



Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного
образовательного учреждения высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИУК Информатика и управление

КАФЕДРА ИУКЗ Системы автоматического управления

ОТЧЕТ
по НАУЧНО-ИССЛЕДОВАТЕЛЬСКОЙ РАБОТЕ
НА ТЕМУ:

Исследование численных методов моделирования систем
автоматического управления и разработка вычислительного
ядра

Студент группы ИУКЗ-71Б

Исаев А.А.
(Подпись) (И.О. Фамилия)

Руководитель НИР

Макаренков А.М.
(Подпись) (И.О. Фамилия)

Оценка руководителя НИР

Баллов, 30-50 (Дата)

Оценка за защиту

Баллов, 30-50 (Дата)

Итоговая оценка

Баллов (по пятибалльной шкале)

Комиссия: _____

Калуга, 2026

**Министерство науки и высшего образования Российской Федерации
Калужский филиал федерального государственного автономного образовательного
учреждения высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(КФ МГТУ им. Н.Э. Баумана)**

УТВЕРЖДАЮ
Заведующий кафедрой ИУКЗ
_____ (Д.В. Мельников)
«___» _____ 2026 г.

З А Д А Н И Е

на научно-исследовательскую работу на тему:

Исследование численных методов моделирования систем автоматического
управления и разработка вычислительного ядра

Задание:

1. Провести анализ существующих программных средств моделирования систем автоматического управления, в том числе отечественных.
2. Описать математические модели типовых звеньев и способ перехода от структурной схемы к системе дифференциальных уравнений.
3. Рассмотреть явные численные методы решения обыкновенных дифференциальных уравнений: метод Эйлера, метод Рунге–Кутты 4-го порядка и метод с автоматическим выбором шага RK45.
4. Разработать вычислительное ядро с консольным запуском и автоматизированной проверкой расчётов.
5. Провести вычислительные эксперименты (точность, устойчивость, время расчёта), обобщить результаты и определить направления дальнейшей работы.

Дата выдачи задания «___» _____ 2026 г.

Руководитель

Макаренков А.М.

(подпись, дата)

(И.О. Фамилия)

Студент

Исаев А.А.

(подпись, дата)

(И.О. Фамилия)

Содержание

Введение.....	4
1. Сбор и анализ материала, анализ литературы.....	6
1.1. Актуальность темы исследования.....	6
1.2. Анализ существующих средств моделирования.....	6
1.3. Обзор численных методов решения дифференциальных уравнений.....	7
1.4. Объект, предмет, цель и задачи исследования.....	8
2. Проведение научного исследования, расчётов.....	10
2.1. Математические модели типовых звеньев.....	10
2.2. Переход от структурной схемы к системе уравнений.....	11
2.3. Реализация вычислительного ядра.....	12
2.4. Методика вычислительного эксперимента.....	14
2.5. Результаты: точность в зависимости от шага.....	14
2.6. Результаты: шаг, необходимый для заданной точности.....	16
2.7. Результаты: устойчивость метода Эйлера.....	17
2.8. Результаты: время расчёта.....	18
2.9. Автоматизированная проверка вычислительного ядра.....	18
3. Обобщение полученных результатов.....	19
3.1. Научные и практические результаты.....	19
3.2. Ограничения и направления дальнейшей работы.....	19
Заключение.....	21
Список использованных источников.....	22
Приложение А. Листинги программы.....	23

Введение

Научно-исследовательская работа выполнялась по направлению подготовки 27.03.04 «Управление в технических системах» и является продолжением работы над выпускной квалификационной работой «Веб-ориентированная система визуального моделирования систем автоматического управления».

В предыдущей части работы был создан прототип веб-системы: пользователь собирает в браузере структурную схему из типовых звеньев и получает график переходного процесса. Чтобы такой системе можно было доверять, нужно убедиться, что её вычислительная часть считает правильно, понять, какой численный метод и с каким шагом использовать, и уметь проверять расчёт без графического интерфейса. Этим вопросам посвящена настоящая НИР.

Цель работы — исследовать численные методы моделирования систем автоматического управления и разработать на их основе вычислительное ядро, результаты которого проверены на примерах с известным решением.

Для достижения цели решались следующие задачи:

- проанализировать существующие программные средства моделирования САУ, включая отечественные;
- описать математические модели типовых звеньев и способ перехода от структурной схемы к системе дифференциальных уравнений;
- рассмотреть метод Эйлера, метод Рунге–Кутты 4-го порядка (RK4) и метод с автоматическим выбором шага (RK45);
- реализовать вычислительное ядро и консольные сценарии проверки;
- провести вычислительные эксперименты: точность от шага, наибольший допустимый шаг, устойчивость метода и время расчёта;
- обобщить результаты и определить направления дальнейшей работы.

Структура отчёта соответствует этапам НИР: сбор и анализ материала, проведение исследования и расчётов, обобщение полученных результатов.

1. Сбор и анализ материала, анализ литературы

1.1. Актуальность темы исследования

В теории автоматического управления систему удобно изображать структурной схемой: каждое звено описывается простым уравнением, а стрелки показывают, как сигнал проходит от входа к выходу. По такой схеме изучают переходные процессы, влияние обратной связи и параметров регулятора на качество регулирования [2, 3].

Вручную получить переходный процесс удаётся только для простых схем. Для остальных используют программы моделирования, которые по схеме составляют систему дифференциальных уравнений и решают её численно. Самым распространённым средством долгое время был пакет MATLAB/Simulink. В настоящее время его поставки российским организациям прекращены, и вузам требуется замена, доступная без лицензии и установки.

Отечественные пакеты SimInTech и Engее решают эту задачу, но являются коммерческими продуктами с закрытым кодом. Для учебного процесса кафедры полезна открытая среда, в которой видно, какую модель программа построила по схеме и каким методом её решила. Ключевая часть такой среды — вычислительное ядро, поэтому выбор и проверка численных методов являются актуальной задачей.

1.2. Анализ существующих средств моделирования

Рассмотрены наиболее известные средства блочного моделирования. Их сравнение приведено в таблице 1.

Таблица 1 – Сравнение средств моделирования динамических систем

Средство	Доступность в РФ	Особенности	Ограничения для учебного применения
MATLAB / Simulink	Поставки прекращены	Большая библиотека блоков, развитые решатели	Лицензия, установка, недоступность
Scilab / Xcos	Свободное ПО	Блочный редактор, открытый код	Установка, устаревший интерфейс

OpenModelica	Свободное ПО	Уравнения на языке Modelica	Высокий порог входа, текстовое описание
SimInTech	Отечественный, коммерческий	Структурные схемы, промышленное применение	Лицензия, закрытый код
Enggee	Отечественный, облачный	Работа в браузере, совместимость с Simulink	Регистрация, коммерческие условия
Разрабатываемая система	Открытая	Браузер, видна собранная модель и метод	Только линейные непрерывные звенья

Во всех рассмотренных средствах расчёт выполняется одинаково: схема превращается в систему обыкновенных дифференциальных уравнений, которая решается численным методом с шагом по времени. От выбранного метода и шага зависят и точность графика, и время расчёта, а при неудачном шаге расчёт может «разойтись» даже для устойчивой системы. Поэтому в работе основное внимание уделено именно численным методам.

1.3. Обзор численных методов решения дифференциальных уравнений

Задача моделирования формулируется как задача Коши: известно начальное состояние $x(0)$ и правило, по которому состояние меняется во времени, — производная $dx/dt = f(t, x)$. Требуется найти $x(t)$ на интервале $[0, t_{\text{end}}]$. Численный метод делит интервал на шаги длиной h и последовательно вычисляет приближения $x_0, x_1, x_2, \dots$ в моменты $t_0, t_1 = t_0 + h, \dots$ [4, 5].

Метод Эйлера — самый простой: следующее значение получают из текущего, двигаясь по касательной:

$$x_{k+1} = x_k + h \cdot f(t_k, x_k). \quad (1)$$

Ошибка метода Эйлера на всём интервале пропорциональна шагу h : при уменьшении шага вдвое она уменьшается примерно вдвое. Такой метод называют методом первого порядка.

Метод Рунге–Кутты 4-го порядка (RK4) на каждом шаге вычисляет четыре пробных значения производной — в начале шага, дважды в середине и в конце — и берёт их взвешенное среднее:

$$k_1 = f(t_k, x_k), \quad k_2 = f(t_k + h/2, x_k + h \cdot k_1/2), \quad (2)$$

$$k_3 = f(t_k + h/2, x_k + h \cdot k_2/2), \quad k_4 = f(t_k + h, x_k + h \cdot k_3), \quad (3)$$

$$x_{k+1} = x_k + h \cdot (k_1 + 2k_2 + 2k_3 + k_4)/6. \quad (4)$$

Ошибка RK4 пропорциональна h^4 : при уменьшении шага вдвое она уменьшается примерно в 16 раз. За это приходится платить четырьмя вычислениями правой части на шаг вместо одного.

Метод RK45 (Дорманда–Принса) на каждом шаге считает решение двумя формулами разного порядка и по их разнице оценивает ошибку. Если ошибка больше заданной, шаг уменьшается, если меньше — увеличивается. В ядре используется реализация `solve_ivp` из библиотеки SciPy с относительной точностью 10^{-8} [6].

У явных методов есть ограничение на шаг. Для звена первого порядка с постоянной времени T метод Эйлера на каждом шаге умножает отклонение от установившегося значения на $(1 - h/T)$. Если $|1 - h/T| > 1$, то есть $h > 2T$, отклонение растёт, и численное решение расходится, хотя реальное звено устойчиво. У RK4 допустимый шаг больше: для того же звена $h < 2,785 \cdot T$. Эти свойства проверяются в главе 2.

1.4. Объект, предмет, цель и задачи исследования

Объектом исследования являются численные методы моделирования линейных непрерывных систем автоматического управления, заданных структурными схемами.

Предметом исследования являются точность, устойчивость и вычислительные затраты методов Эйлера, RK4 и RK45 при моделировании типовых звеньев и замкнутых контуров.

Цель и задачи исследования сформулированы во введении. Результатом должно стать вычислительное ядро, которое запускается из

консоли без графического интерфейса и сравнивает каждый метод с точным решением.

2. Проведение научного исследования, расчётов

2.1. Математические модели типовых звеньев

Вычислительное ядро поддерживает типовые звенья, приведённые в таблице 2. Каждое звено описывается передаточной функцией $W(s)$, где s — оператор дифференцирования. Динамические звенья добавляют в модель переменные состояния, безынерционные — нет.

Таблица 2 – Типовые звенья вычислительного ядра

Звено	Передаточная функция / уравнение	Состояний
Ступенчатый вход	$u(t) = A$ при $t \geq t_0$, иначе 0	0
Усилитель	$y = k \cdot u$	0
Сумматор	$y = \pm u_1 \pm u_2 \pm \dots$	0
Интегратор	$W(s) = k / s$	1
Апериодическое звено 1-го порядка	$W(s) = K / (T \cdot s + 1)$	1
Колебательное звено	$W(s) = K \cdot \omega_0^2 / (s^2 + 2\zeta\omega_0 \cdot s + \omega_0^2)$	2
Передаточная функция	$W(s) = N(s) / D(s)$	deg D
ФНЧ Баттерворта	$ W(j\omega) ^2 = 1 / (1 + (\omega/\omega_c)^{2n})$	n
ПИД-регулятор	$W(s) = K_p + K_i/s + K_d \cdot N \cdot s / (s + N)^*$	2
Осциллограф	фиксирует выходной сигнал	0

* В дифференциальной части используется фильтр, чтобы звено было физически реализуемым.

Переходные характеристики (реакция на единичную ступеньку) основных звеньев показаны на рисунке 1. Интегратор растёт линейно, звено 1-го порядка плавно выходит на значение K , колебательное звено совершает затухающие колебания, а фильтр Баттерворта 4-го порядка выходит на установившееся значение с небольшим перерегулированием.

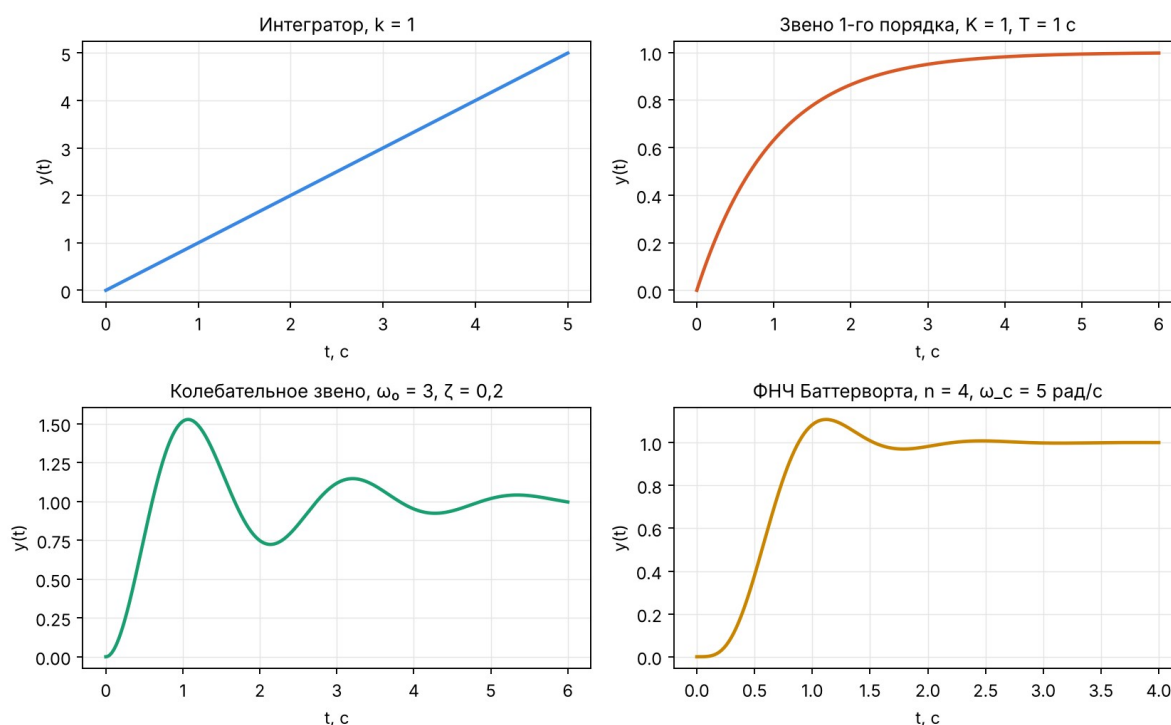


Рисунок 1 – Переходные характеристики типовых звеньев

2.2. Переход от структурной схемы к системе уравнений

Путь от рисунка к графику показан на рисунке 2. Сначала схема проверяется: у каждого входа должен быть ровно один источник, параметры должны быть допустимыми (например, $T > 0$), связи — ссылаться на существующие порты. Затем ядро собирает модель всей схемы в пространстве состояний:

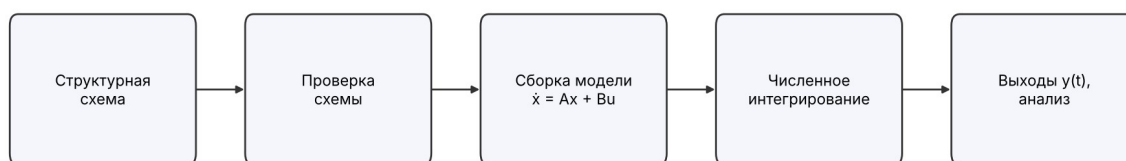


Рисунок 2 – Этапы расчёта в вычислительном ядре

$$\frac{dx}{dt} = A \cdot x + B \cdot u, \quad y = C \cdot x + D \cdot u, \quad (5)$$

где x — переменные состояния всех динамических звеньев, u — входные воздействия, y — сигналы осциллографов, A , B , C , D — матрицы, которые программа строит автоматически по связям схемы. Пользователь видит эти матрицы на вкладке «Матрицы».

Например, для схемы с обратной связью на рисунке 3 объект $1/(s - 1)$ сам по себе неустойчив, но вместе с усилителем $K = 3$ и отрицательной обратной связью получается замкнутая система $dx/dt = -2 \cdot x + 3 \cdot u$ с выходом $y = x$. Её переходный процесс стремится к значению 1,5.

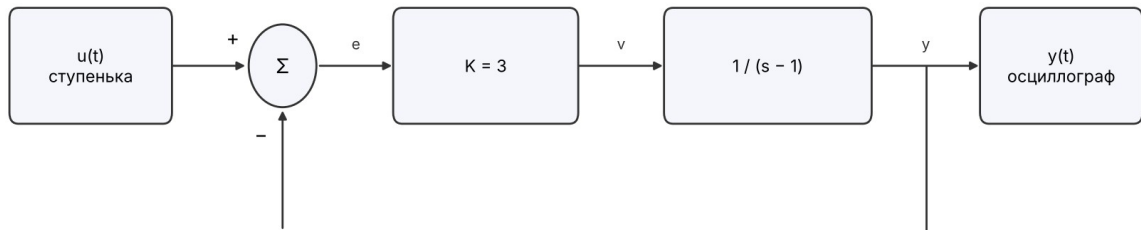


Рисунок 3 – Схема «Стабилизация неустойчивого объекта»

После сборки модели правая часть $f(t, x) = A \cdot x + B \cdot u(t)$ передаётся численному методу. Ступенчатый вход меняется скачком, поэтому момент скачка t_0 всегда включается в сетку времени: шаг заканчивается ровно в момент скачка, и следующий шаг уже видит новое значение входа.

Модель схемы, собранная программой, отображается в интерфейсе (рисунок 4).

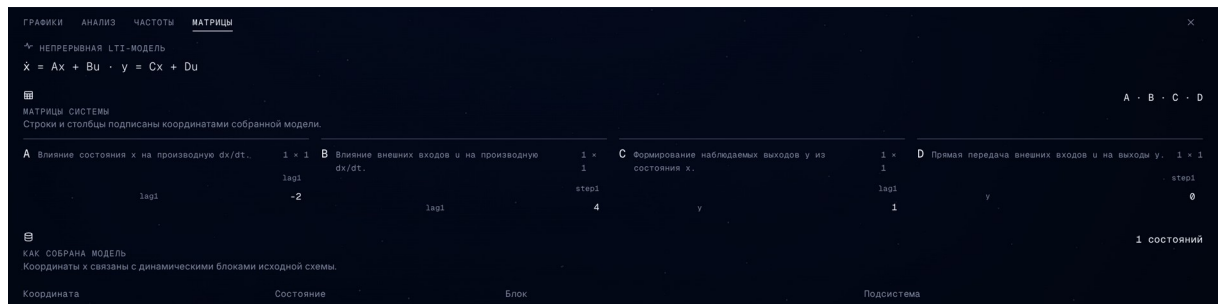


Рисунок 4 – Матрицы модели звена 1-го порядка на вкладке «Матрицы»

2.3. Реализация вычислительного ядра

Вычислительное ядро написано на Python 3 с использованием библиотек NumPy и SciPy [6, 7] и не зависит от графического интерфейса. Его можно вызвать из веб-сервера (FastAPI) или напрямую из консольного сценария. Место ядра в системе показано на рисунке 5.



Рисунок 5 – Место вычислительного ядра в программной системе

Основные модули ядра перечислены в таблице 3.

Таблица 3 – Модули вычислительного ядра

Модуль	Назначение
app/core/block_specs.py	описание типовых звеньев, портов и допустимых параметров
app/validation	проверка структурной схемы до расчёта
app/simulation/hierarchy.py	раскрытие подсистем в плоскую схему
app/simulation/assembly.py	сборка матриц A, B, C, D по связям схемы
app/simulation/solvers.py	численные методы: Эйлер, RK4, RK45, точное решение
app/simulation/service.py	единый сценарий расчёта для веб-сервера
scripts/nir_solver_study.py	консольное исследование методов (данная НИР)
app/tests	автоматизированные тесты ядра (pytest)

Все три метода имеют одинаковый вызов: правая часть, начальное состояние и сетка времени. Это позволяет сравнивать их в одном цикле. Реализация метода Эйлера приведена в листинге 1 приложения А, метода RK4 — в листинге 2. Эталоном для сравнения служит точное решение линейной системы через матричную экспоненту $x(t + h) = e^{Ah} \cdot x(t) + (\int_0^h e^{A\tau} d\tau) \cdot B \cdot u$, которое не содержит ошибки численного метода, кроме ошибок округления (около 10^{-12}).

Консольный сценарий запускается одной командой:

```
cd backend
python scripts/nir_solver_study.py
```

Сценарий строит три тестовые схемы, рассчитывает их всеми методами при разных шагах, сравнивает с точным решением и сохраняет таблицы (docs/study/nir_*.csv), графики (docs/figures/nir_*.png) и сводный отчёт

(docs/nir_solver_study.md). Повторный запуск даёт те же числа, кроме времени расчёта, которое зависит от компьютера.

2.4. Методика вычислительного эксперимента

Для экспериментов выбраны три схемы с известным решением:

- апериодическое звено 1-го порядка, $K = 1$, $T = 1$ с; точное решение $y(t) = 1 - e^{-t}$;
- колебательное звено, $\omega_0 = 2$ рад/с, $\zeta = 0,3$ — затухающие колебания;
- замкнутый контур: ПИД-регулятор ($K_p = 2$, $K_i = 1$) и колебательный объект с отрицательной обратной связью — система третьего порядка.

Интервал моделирования — 8 с, входное воздействие — единичная ступенька в момент $t = 0$. Шаг h уменьшался вдвое: 0,2; 0,1; 0,05; 0,025; 0,0125 с. Для каждого метода вычислялась наибольшая ошибка на интервале:

$$\varepsilon(h) = \max_k |y_{\text{метод}}(t_k) - y_{\text{точн}}(t_k)|. \quad (6)$$

Наблюдаемый порядок метода определялся по отношению ошибок при соседних шагах:

$$p = \log_2(\varepsilon(h) / \varepsilon(h/2)). \quad (7)$$

Для метода первого порядка $p \approx 1$, для метода четвёртого порядка $p \approx 4$.

2.5. Результаты: точность в зависимости от шага

Результаты расчёта ошибок приведены в таблице 4, графически — на рисунке 6 (логарифмический масштаб по обеим осям).

Таблица 4 – Наибольшая ошибка методов при разных шагах

Схема	h, с	Эйлер	RK4	RK45	p Эйлер	p RK4
звено 1-го порядка	0,2	4.02e-02	5.80e-06	1.84e-09	—	—
звено 1-го порядка	0,1	1.92e-02	3.33e-07	1.90e-09	1,07	4,12
звено 1-го порядка	0,05	9.39e-03	2.00e-08	1.91e-09	1,03	4,06
звено 1-го порядка	0,025	4.65e-03	1.22e-09	1.91e-09	1,02	4,03
звено 1-го порядка	0,0125	2.31e-03	7.56e-11	1.92e-09	1,01	4,02
колебательное звено	0,2	4.01e-01	2.98e-04	3.61e-09	—	—
колебательное звено	0,1	1.52e-01	1.78e-05	3.61e-09	1,40	4,06
колебательное звено	0,05	6.95e-02	1.09e-06	3.61e-09	1,13	4,03
колебательное звено	0,025	3.33e-02	6.74e-08	3.61e-09	1,06	4,02

колебательное звено	0,0125	1.63e-02	4.19e-09	3.61e-09	1,03	4,01
замкнутый контур с ПИД	0,2	1.20e+02	3.89e-03	7.35e-09	—	—
замкнутый контур с ПИД	0,1	2.11e+00	2.37e-04	7.35e-09	5,83	4,04
замкнутый контур с ПИД	0,05	2.75e-01	1.46e-05	7.35e-09	2,94	4,02
замкнутый контур с ПИД	0,025	1.06e-01	9.08e-07	7.37e-09	1,38	4,01
замкнутый контур с ПИД	0,0125	4.74e-02	5.65e-08	7.43e-09	1,15	4,01

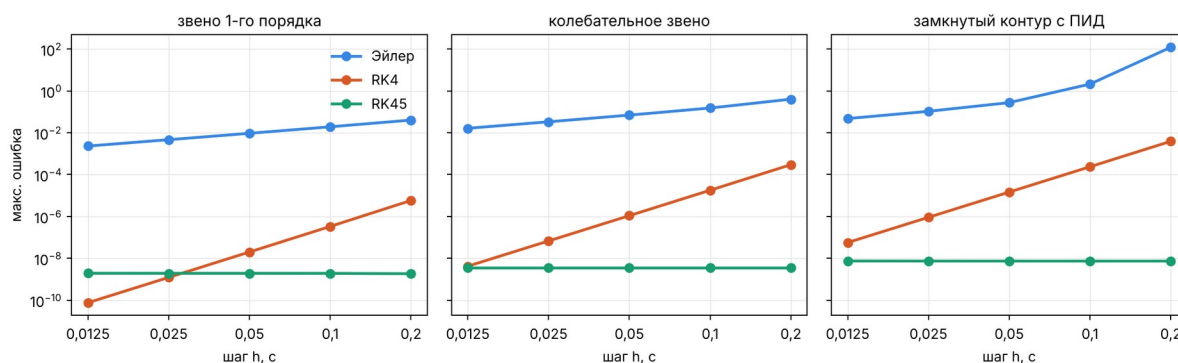


Рисунок 6 – Зависимость ошибки от шага для трёх схем

Из таблицы видно, что при малых шагах наблюдаемый порядок метода Эйлера приближается к 1, а метода RK4 — к 4, что совпадает с теорией. На замкнутом контуре при больших шагах (0,2 и 0,1 с) порядок Эйлера получается больше 1: при таком шаге метод ещё неточен настолько, что решение искажено сильнее, чем предсказывает асимптотическая оценка. Ошибка RK45 почти не зависит от шага сетки вывода, потому что метод сам выбирает внутренний шаг под заданную точность 10^{-8} .

Наглядно различие видно на рисунке 7: при одном и том же шаге $h = 0,1$ с решение методом Эйлера заметно отстаёт от точного и завышает амплитуду колебаний, а точки RK4 лежат на точной кривой.

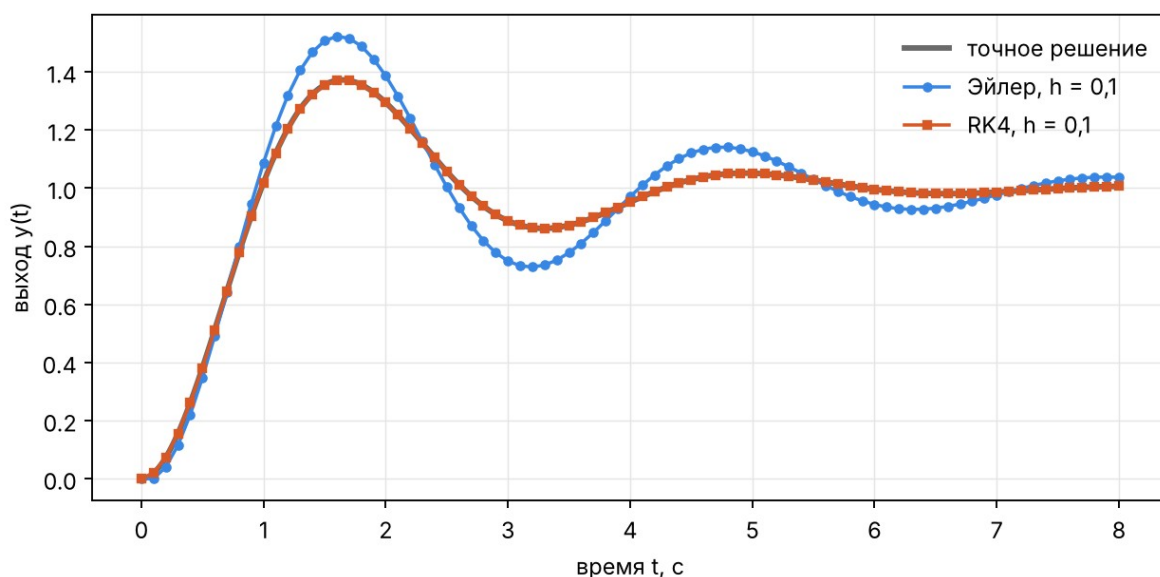


Рисунок 7 – Колебательное звено: метод Эйлера и RK4 при $h = 0,1$ с

2.6. Результаты: шаг, необходимый для заданной точности

Для практики важно, сколько шагов нужно методу, чтобы ошибка не превышала заданную величину. Для допуска 10^{-3} шаг уменьшался вдвое, начиная с 0,5 с, пока ошибка не становилась меньше допуска. Результаты приведены в таблице 5.

Таблица 5 – Шаг и число шагов для ошибки не более 10^{-3} на интервале 8 с

Схема	h Эйлер, с	шагов Эйлер	h RK4, с	шагов RK4	Выигрыш RK4, раз
звено 1-го порядка	0,00390625	2048	0,5	16	128
колебательное звено	0,00048828 1	16384	0,25	32	512
замкнутый контур с ПИД	0,00024414 1	32768	0,125	64	512

Методу Эйлера требуется в 128–512 раз больше шагов, чем RK4. Даже с учётом того, что шаг RK4 в четыре раза дороже, метод RK4 оказывается в десятки раз выгоднее. Поэтому метод Эйлера в вычислительном ядре используется только как учебный эталон, а для расчётов пользователю предлагаются RK4 и RK45.

2.7. Результаты: устойчивость метода Эйлера

Для звена 1-го порядка с $T = 1$ с выполнялся расчёт на интервале 30 с с разными шагами. Оценивалось отклонение выхода от установившегося значения 1 в конце интервала. Результаты приведены в таблице 6 и на рисунке 8.

Таблица 6 – Поведение методов при больших шагах (звено 1-го порядка, $T = 1$ с)

h / T	Множитель $ 1 - h/T $	Эйлер: отклонение	Эйлер	RK4: отклонение	RK4
0,5	0,5	0.0e+00	сходится	9.6e-14	сходится
1,0	0,0	0.0e+00	сходится	1.7e-13	сходится
1,5	0,5	9.5e-07	сходится	5.5e-12	сходится
1,9	0,9	1.9e-01	сходится	5.6e-09	сходится
2,1	1,1	3.8e+00	расходится	9.7e-07	сходится
2,5	1,5	1.3e+02	расходится	5.5e-03	сходится

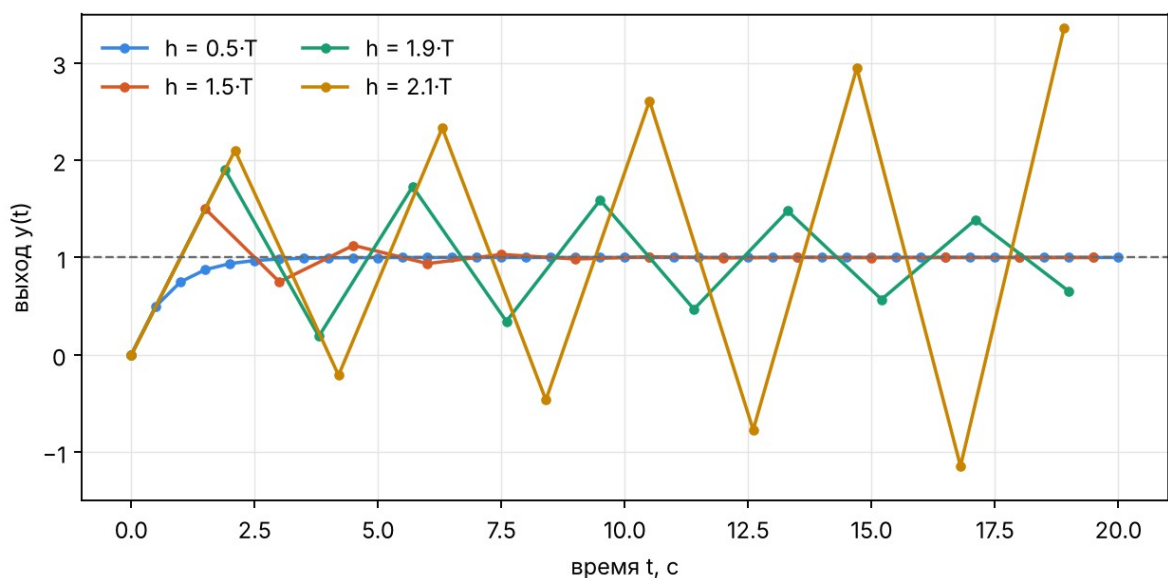


Рисунок 8 – Решение методом Эйлера при разных шагах

При $h < T$ решение монотонно выходит на 1. При $T < h < 2T$ появляются численные колебания, которых нет у реального звена, но они затухают. При $h > 2T$ колебания нарастают, и расчёт расходится. Метод RK4 при тех же шагах остаётся устойчивым, поскольку его допустимый шаг для этого звена больше ($h < 2,785 \cdot T$).

Вывод для пользователя простой: слишком большой шаг может дать неверный, «раскачивающийся» график даже для устойчивой системы. Поэтому вычислительное ядро перед расчётом методом RK4 проверяет шаг по полюсам модели и при необходимости сообщает наибольший допустимый шаг.

2.8. Результаты: время расчёта

Время расчёта замкнутого контура на интервале 10 с с шагом 0,01 с приведено в таблице 7 (медиана 15 запусков).

Таблица 7 – Время расчёта и ошибка методов при $h = 0,01$ с

Метод	Вычислений f на шаг	Время, мс	Ошибка
Эйлер	1	7,45	3.7e-02
RK4	4	23,41	2.3e-08
RK45	переменно	19,28	7.4e-09

Метод Эйлера самый быстрый, но его ошибка на 6 порядков больше, чем у RK4. Время расчёта всех методов составляет десятки миллисекунд, что незаметно для пользователя веб-системы. Следовательно, для учебных схем выбор метода определяется точностью, а не скоростью.

2.9. Автоматизированная проверка вычислительного ядра

Правильность ядра проверяется автоматизированными тестами `pytest`, которые запускаются одной командой. Для численных методов проверяется: совпадение RK45 с точным решением, четвёртый порядок RK4, первый порядок метода Эйлера, точное значение одного шага метода Эйлера для уравнения $dx/dt = -x$ и сообщение об ошибке при расхождении расчёта. Тесты метода Эйлера приведены в листинге 4 приложения А. Всего набор тестов серверной части включает более 200 проверок, все они выполняются успешно.

3. Обобщение полученных результатов

3.1. Научные и практические результаты

В ходе НИР получены следующие результаты:

- проведён обзор средств моделирования САУ и обоснована потребность в открытой учебной среде в условиях недоступности MATLAB/Simulink;
- описаны модели десяти типовых звеньев и способ автоматического перехода от структурной схемы к системе уравнений $dx/dt = A \cdot x + B \cdot u$;
- реализованы и проверены три численных метода: Эйлер, RK4 и RK45;
- экспериментально подтверждены теоретические порядки точности: около 1 для метода Эйлера и около 4 для RK4;
- показано, что для ошибки 10^{-3} методу Эйлера нужно в сотни раз больше шагов, чем RK4;
- подтверждена граница устойчивости метода Эйлера $h = 2T$: при большем шаге расчёт расходится;
- разработан консольный сценарий исследования, который воспроизводит все таблицы и графики отчёта одной командой.

Практический результат — вычислительное ядро, которое используется веб-системой визуального моделирования. По результатам исследования в системе по умолчанию используется метод RK45, а метод RK4 доступен с проверкой допустимого шага.

3.2. Ограничения и направления дальнейшей работы

Текущая версия ядра имеет ограничения:

- поддерживаются только линейные непрерывные звенья, нет нелинейных элементов (насыщение, зона нечувствительности, реле);
- входные воздействия — только ступенчатые;
- не реализованы неявные методы для жёстких систем.

В рамках выпускной квалификационной работы планируется развитие веб-системы: модуль показателей качества переходного процесса, частотный анализ, хранение проектов и многоуровневые схемы. Возможное продолжение исследования — нелинейные звенья и неявные методы интегрирования.

Заключение

В результате выполнения научно-исследовательской работы исследованы численные методы моделирования систем автоматического управления и разработано вычислительное ядро веб-системы визуального моделирования.

Проведён анализ существующих средств моделирования, описаны модели типовых звеньев и способ получения системы дифференциальных уравнений по структурной схеме. Рассмотрены метод Эйлера, метод Рунге–Кутты 4-го порядка и метод RK45 с автоматическим выбором шага.

Вычислительные эксперименты на схемах с известным решением подтвердили теоретические порядки точности методов и границу устойчивости метода Эйлера. Показано, что метод RK4 при одинаковой точности требует в сотни раз меньше шагов, чем метод Эйлера, а время расчёта учебных схем любым методом составляет десятки миллисекунд.

Разработанное ядро запускается из консоли, проверено автоматизированными тестами и используется в веб-системе визуального моделирования, которая является предметом выпускной квалификационной работы.

Список использованных источников

1. Финошин А.В., Корнюшин Ю.П., Краснощеченко В.И. Научно-исследовательская работа: практикум. – Калуга: КФ МГТУ им. Н.Э. Баумана, 2024. – 17 с.
2. Бесекерский В.А., Попов Е.П. Теория систем автоматического регулирования. – М.: Директмедиа, 2021. – 747 с.
3. Ким Д.П. Теория автоматического управления. Т. 1. Линейные системы. – М.: Физматлит, 2016. – 312 с.
4. Самарский А.А., Гулин А.В. Численные методы: учеб. пособие для вузов. – М.: Наука, 1989. – 432 с.
5. Hairer E., Nørsett S.P., Wanner G. Solving Ordinary Differential Equations I: Nonstiff Problems. – Berlin; Heidelberg: Springer, 2008. – 528 p.
6. SciPy API Reference: `scipy.integrate.solve_ivp` [Электронный ресурс]. – URL: https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html (дата обращения: 01.10.2026).
7. NumPy Documentation [Электронный ресурс]. – URL: <https://numpy.org/doc/> (дата обращения: 01.10.2026).
8. Горелов Ю.Н., Курганская Л.В. Численные методы решения обыкновенных дифференциальных уравнений (метод Рунге–Кутты): учебное пособие. – Самара: Изд-во Самарского университета, 2023. – 76 с.

Приложение А. Листинги программы

Листинг 1 – Метод Эйлера (app/simulation/solvers.py)

```
def euler_integrate(
    rhs: Rhs,
    x0: np.ndarray,
    t_grid: np.ndarray,
    *,
    breakpoints: Sequence[float] = (),
) -> np.ndarray:
    """Explicit Euler on the given grid:  $x_{k+1} = x_k + h \cdot f(t_k, x_k)$ .

    First-order reference method for the numerical study; it is not offered by
    the API. The derivative is taken at the left end of each step, so a jump at
    a breakpoint grid node takes effect from the step that starts there.
    """

    _validate_time_grid(t_grid)
    del breakpoints # the left-point rule never evaluates past the step start
    if x0.size == 0:
        return np.zeros((0, t_grid.size))

    trajectory = np.zeros((x0.size, t_grid.size))
    trajectory[:, 0] = x0
    x = x0.astype(float).copy()
    for index in range(t_grid.size - 1):
        left, right = float(t_grid[index]), float(t_grid[index + 1])
        with np.errstate(over="ignore", invalid="ignore"): # divergence is reported
            below
                x = x + (right - left) * rhs(left, x)
            if not np.all(np.isfinite(x)):
                raise SolverError(f"Эйлер: состояние перестало быть конечным в момент
t={right:g}.")
            trajectory[:, index + 1] = x
    return trajectory
```

Листинг 2 – Метод Рунге–Кутты 4-го порядка (app/simulation/solvers.py)

```
def rk4_integrate(
    rhs: Rhs,
    x0: np.ndarray,
    t_grid: np.ndarray,
    *,
    breakpoints: Sequence[float] = (),
) -> np.ndarray:
    """Classical RK4 on the given grid; every breakpoint must be a grid node.

    At a step that ends exactly on a breakpoint the last stage is evaluated
    just before it (left limit of the input), so the jump is applied only at
    the start of the next step.
    """

    _validate_time_grid(t_grid)
    if x0.size == 0:
        return np.zeros((0, t_grid.size))

    ends_on_jump = np.isin(t_grid[1:], np.asarray(breakpoints, dtype=float))
    trajectory = np.zeros((x0.size, t_grid.size))
    trajectory[:, 0] = x0
    x = x0.astype(float).copy()
    for index in range(t_grid.size - 1):
        left, right = float(t_grid[index]), float(t_grid[index + 1])
        h = right - left
        k4_time = float(np.nextafter(right, left)) if ends_on_jump[index] else right
        k1 = rhs(left, x)
        k2 = rhs(left + 0.5 * h, x + 0.5 * h * k1)
```

```

k3 = rhs(left + 0.5 * h, x + 0.5 * h * k2)
k4 = rhs(k4_time, x + h * k3)
x = x + (h / 6.0) * (k1 + 2.0 * k2 + 2.0 * k3 + k4)
if not np.all(np.isfinite(x)):
    raise SolverError(f"RK4: состояние перестало быть конечным в момент
t={right:g}.")
trajectory[:, index + 1] = x
return trajectory

```

Листинг 3 – Расчёт ошибки и наблюдаемого порядка

(scripts/nir_solver_study.py)

```

def accuracy_versus_step() -> tuple[list[dict], dict[str, dict[str, list[float]]]]:
    rows, curves = [], {}
    for case, diagram in CASES.items():
        model = model_of(diagram)
        errors: dict[str, list[float]] = {name: [] for name in METHODS}
        for dt in STEPS:
            grid = grid_of(8.0, dt)
            reference = exact(model, grid)
            row = {"схема": case, "шаг h": dt}
            for name, method in METHODS.items():
                error = float(np.max(np.abs(simulate(model, method, grid) -
reference)))
                errors[name].append(error)
                row[f"ошибка {name}"] = f"{error:.2e}"
            rows.append(row)
        for name in ("Эйлер", "RK4"):
            orders = np.log2(np.asarray(errors[name][:-1]) / np.asarray(errors[name]
[1:]))
            for row, order in zip(rows[-len(STEPS) + 1:], orders, strict=True):
                row[f"порядок {name}"] = f"{order:.2f}"
                rows[-len(STEPS)][f"порядок {name}"] = "-"
        curves[case] = errors
    return rows, curves

```

Листинг 4 – Тесты метода Эйлера (app/tests/test_solvers.py)

```

def test_euler_has_first_order_convergence() -> None:
    model = _model(pid_loop_case().diagram)
    errors = []
    for dt in (0.004, 0.002, 0.001):
        grid = _grid(model, 4.0, dt)
        exact = _outputs(model, grid, exact_lti_integrate(model.a, model.b,
model.source_values, model.x0, grid))
        euler = _outputs(model, grid, euler_integrate(model.rhs, model.x0, grid))
        errors.append(float(np.max(np.abs(euler - exact))))

    orders = np.log2(np.asarray(errors[:-1]) / np.asarray(errors[1:]))
    assert np.all((orders > 0.9) & (orders < 1.1)), orders

def test_euler_step_is_the_left_point_rule() -> None:
    # x' = -x, x(0) = 1: one step of size h gives exactly 1 - h.
    trajectory = euler_integrate(lambda _t, x: -x, np.array([1.0]), np.array([0.0,
0.25]))
    assert trajectory[0, 1] == pytest.approx(0.75, abs=1e-15)

def test_euler_beyond_its_stability_limit_is_reported() -> None:
    # x' = -x with h = 3 > 2: |1 - h| = 2, the numerical solution doubles every step.
    grid = np.arange(0.0, 3.0 * 1100, 3.0)
    with pytest.raises(SolverError, match="Эйлер"):
        euler_integrate(lambda _t, x: -x, np.array([1.0]), grid)

```

Листинг 5 – Сценарий расчёта схемы (app/simulation/service.py, фрагмент)

```
def simulate_request(request: SimulationRequest) -> SimulationResponse:
    compiled = compile_model(request.diagram)
    model = compiled.model
    t_eval = build_time_grid(request)
    breakpoints = model.step_breakpoints(float(t_eval[0]), float(t_eval[-1]))
    warnings: list[str] = []

    trajectory = _integrate(model, request.solver, t_eval, breakpoints, warnings)
    sources = np.stack([model.source_values(float(t)) for t in t_eval], axis=1) if
model.sources \
        else np.zeros((0, t_eval.size))
    readings = model.c @ trajectory + model.d @ sources
    outputs = {scope.label: readings[index].tolist() for index, scope in
enumerate(model.scopes)}

    system_analysis = analyze_model(model)
    stability = system_analysis["stability"]
    steady = steady_state_outputs(model, stability)
    steady_values = {
        scope.label: (None if steady is None else float(steady[index]))
        for index, scope in enumerate(model.scopes)
    }
    unavailable_reason = None
    if steady is None:
        unavailable_reason = (
            "Установившееся значение не существует: система неустойчива."
            if stability == "unstable"
            else "Установившееся значение не определено: система на границе
устойчивости."
        )
    elif any(s.amplitude != 0.0 and s.t0 >= float(t_eval[-1]) for s in model.sources):
```

Листинг 6 – Результат автоматизированной проверки

```
$ cd backend
$ python -m pytest -q app/tests/test_solvers.py
..... [100%]
11 passed in 0.41s
```